

TECHNICAL LOGIC & DESIGN BLUEPRINT: PROJECT ZENITH

Event: Aaruush '26 - AstralWeb Innovate | **Track:** Real-Time Cosmic Radar Implementation

1. Executive Summary & Concept Originality

Project Zenith introduces an inverted perspective to orbital tracking by creating a ground-up sensory radar dome mapped to localized Earth coordinates. Instead of utilizing standard macro 3D globe representations, the application functions as a highly targeted, responsive cosmic radar. It isolates and displays celestial assets, active payloads, and orbital debris passing through the user's explicit local hemisphere.

To achieve maximum synchronization speeds without UI rendering stutter, the architecture employs client-side orbital propagation instead of relying on continuous server requests. The core of this system is an interactive, **Modular Multi-Lens Dashboard Layout** that consolidates **seven distinct utility lenses**—**Live Zenith Radar, Location Picker, Object Info Panel, Zenith Time Machine, Debris Lens, Cosmic History Cards, and Sky Visibility Score**—into a unified workspace optimized for high-performance predictive timeline analytics.

2. Data Strategy & Asynchronous Pipeline

To eliminate live runtime network lag, Project Zenith uses an optimized, multi-tiered data pipeline built on free, open-source telemetry endpoints:

- **TLE Ingestion (CelesTrak API):** Upon user initialization, the client executes a single background fetch to retrieve global Two-Line Element (TLE) orbital matrix data streams. This raw data is parsed and cached locally inside the browser's native IndexedDB storage, eliminating repetitive client-server requests.
- **Atmospheric Metrics (Open-Meteo API):** The frontend triggers a lightweight, keyless API request to pull real-time localized cloud coverage indices based on coordinates chosen via the Location Picker.
- **Bortle Sky Scale Vectors:** Coordinates are matched against a static JSON grid of global artificial night sky brightness data to output light pollution parameters.
- **Deep Space Coordinates (NASA Horizons API):** The platform fetches state vectors for major planetary bodies to enable precise directional alignment mapping across the local observer window.

3. Proposed Technical Stack

The decoupled application stack ensures a smooth 60 FPS performance envelope during high-frequency mathematical coordinate computation loops:

System Layer	Selected Technology	Operational Role & Component Mapping
Frontend Layout	Next.js 14 + React	Coordinates global application state, handling layout state management fluidly across modular UI lens shifts.
Geospatial Canvas	CesiumJS + WebGL	Drives the interactive 3D map for the Location Picker and renders the polar 2D circle canvas for the Live Zenith Radar .
Physics Engine	satellite.js via Web Workers	Runs local SGP4 propagation. Adjusting the Zenith Time Machine passes a time scalar to this background worker thread.
Local Cache	Browser IndexedDB	Stores raw daily TLE matrices locally, executing lightning-fast structural array filtering for the Debris Lens .

4. Mathematical Framework & Edge Performance

An orbital asset is identified as traversing the user's localized Zenith window when its calculated horizontal elevation angle satisfies the target astronomical boundary condition:

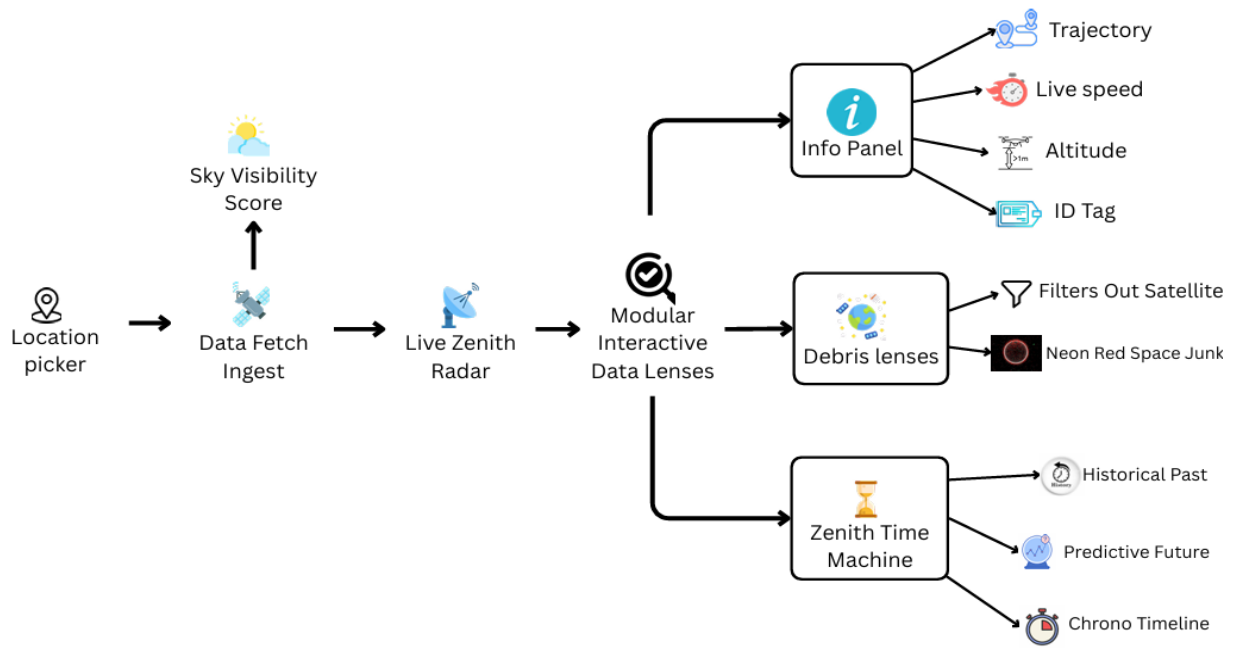
Elevation (E) ≥ 80°

The value is computed programmatically by taking the dot product of the observer's localized upward horizon vector (H) and the relative observer-to-satellite range vector (ρ) derived from Earth-Centered Inertial (ECI) coordinates:

Sin(E) = (H · ρ) / (|H| |ρ|)

Executing this intensive processing on an isolated background browser `Web Worker` thread allows the client to evaluate hundreds of concurrent position vectors every frame without introducing thread blocking or degrading UI input responsiveness.

5. User Flow & Low-Fidelity Layout Matrix



Operational Execution Steps:

Step 1: The user sets position parameters inside the **Location Picker**, while raw TLE sets ingest into local **IndexedDB** cache storage.

Step 2: The polar radar coordinate map resolves, rendering localized asset tracks overhead paired with active **Sky Visibility Score** updates.

Step 3: The user toggles the **Debris Lens** to filter out working components, clicks specific targets to view a **Cosmic History Card**, or scrubs the **Zenith Time Machine** slider to compute predictive future passes.

6. Technical Feasibility & Asset Analysis

- **Thread Offloading Optimization:** Shifting the core mathematical calculation loop entirely inside an asynchronous background **Web Worker thread** prevents intensive calculation matrices from freezing the primary interface thread.
- **Memory Management Optimization:** Running direct local calculations (**Array.prototype.filter()**) against cached local storage elements removes backend dependencies, ensuring reliable operation on low-power mobile or consumer web clients.
- **Zero-Cost Infrastructure Scaling:** All structural modules run on completely open-source foundations (**Next.js**, **CesiumJS**, **satellite.js**, **Open-Meteo**), ensuring complete architectural and execution feasibility at a baseline operating budget of exactly **zero rupees**.